JNEOPALLIUM · INDUSTRIAL LOOP GUARDIAN

ARCHITECTURE ARTICLE

A Supervisory Brain for Your Plant

Cross-loop, cross-timescale diagnosis above PLC/PID/SIS — complete architecture, explained

How a brain-inspired supervisory layer turns the OPC UA and MQTT telemetry you already have into early diagnosis of pump wear, control-loop oscillation, sensor drift, and wasted energy — without ever touching your deterministic controls. Written so a non-specialist can follow every step.

Document	Architecture & Technical Overview
Product	Jneopallium Industrial Loop Guardian (FMI Skid Demo)
Model	industrial-loop-guardian 1.0.0
Safety mode	ADVISORY (supervisory, recommend-only)
Author	Dmytro Rakovskyi — Kharkiv, Ukraine
Date	23 June 2026
Audience	Plant managers, controls engineers, executives, newcomers
License	BSD 3-Clause

What's inside

1 Executive summary

A good PID controller holds one variable — a temperature, a flow, a level — beautifully. But a plant rarely loses money on a single well-tuned loop. It loses money on the things that **no single loop can see**: a pump quietly wearing out, a control loop slowly drifting out of tune, a temperature sensor creeping off-calibration, and a combination of controllers that each behave correctly while together burning more energy than they should.

The **Jneopallium Industrial Loop Guardian** is a supervisory layer that sits **above** your PLC, PID, and safety systems — never instead of them. It reads the OPC UA and MQTT telemetry you already produce, correlates signals across many timescales (milliseconds to weeks), and emits a single, evidenced advisory: what is happening, why, how urgent it is, and what it is worth in money. It never blocks, never trips, never overrides a safety interlock.

> **The one sentence to remember**
>
> Your PLC/PID/SIS keeps doing the fast, deterministic control and hard safety. The Loop Guardian adds cross-loop, cross-timescale operational intelligence on top — diagnosis, energy optimisation, and maintenance planning — and only ever recommends.

Its design is borrowed from the brain: fast reflexes and slow judgement running at the same time, with specialised "neurons" that combine signals other tools look at in isolation. The result is a product that is **easy to trust** — it runs in ADVISORY mode, keeps every deterministic control authoritative, and explains every recommendation.

2 The problem: why good controls still lose money

A single loop is not where the money leaks

Classic controls and threshold alarms look at one variable at a time. That is exactly right for fast, stable regulation — and exactly wrong for the expensive, slow-moving problems:

- **Hidden degradation.** A bearing wears over weeks. "Bearing temperature > 85 °C → alarm" fires only at the end — after the damage. The early evidence is spread across vibration, suction pressure, power-per-unit-flow, and running hours, on different clocks.
- **Cross-loop waste.** A heater raises temperature while a cooling valve removes heat while a pump runs faster than needed. Each local controller is "correct"; the combination wastes energy no single loop can detect.
- **Nuisance alarms.** The same rising temperature is normal at startup and alarming in steady state. Context-free thresholds either over-alarm or miss the real event.
- **Sensor-fault ambiguity.** Is the process actually getting hotter, or is one sensor drifting? Majority voting cannot tell; the answer needs the physical model, the actuator commands, and the maintenance history together.

What that costs

Unplanned shutdowns, unnecessary preventive maintenance, shortened equipment life, and quietly elevated energy bills. Every one of these is a decision that depends on **several signals evolving over different timescales** — precisely the case where a supervisory, multi-timescale architecture earns its keep, and a single PID loop cannot.

3 The big idea: supervise, don't replace

The commercially credible — and safe — design is a hybrid. Keep what already works; add what it cannot do:

```
PLC / PID / SIS
    +-- deterministic millisecond control and hard safety   (unchanged)
Jneopallium Industrial Loop Guardian  (supervisory, above)
    +-- multi-loop coordination
    +-- oscillation and sensor-fault diagnosis
    +-- energy optimisation
    +-- degradation prediction and maintenance planning
    +-- bounded setpoint recommendations (advisory)
```

A PID controller usually **beats** a sophisticated AI on cost, predictability, validation effort, and reliability when controlling one stable variable. So we do not compete there. The Loop Guardian becomes valuable only when a decision depends on many signals over many timescales — diagnosis, economic ranking, and planning.

> **Where it does and does not help**
>
> High value: predictive maintenance, oscillation diagnosis, sensor-drift discrimination, energy and setpoint optimisation, batch-context anomaly detection. Low or no value: a single stable regulatory loop, and hard interlocks / emergency shutdown — which must remain deterministic. The product is designed to respect that line.

4 The platform underneath: the Jneopallium framework

The Loop Guardian is one application of **Jneopallium**, a Java framework for building biologically-grounded neuron networks (published in the *International Journal of Science and Research*, 2024). Three of its ideas explain why it is good at supervisory work.

1. Typed signals — meaning, not just numbers

A flow measurement, a vibration reading, a maintenance work-order, and an energy meter are *different kinds of signal* with different meaning and urgency. The engine routes each to the right specialist and keeps the evidence intact all the way to the advisory.

2. Many clocks — fast reflexes, slow judgement

Biology runs on many clocks at once. Jneopallium copies this: fast signals (measurements, commands, interlocks) are processed every loop; slower context (degradation, efficiency, maintenance windows) is processed every 10 to 60 loops. A pump-health neuron can weigh instantaneous hydraulics against a multi-week wear trend in one coherent judgement.

Signal	Cadence	Function
Flow, pressure, interlocks	Every tick	Immediate hydraulic condition and safety state
Pump command vs. actual speed	Every tick	Actuator tracking
Vibration RMS	Every 5–10 ticks	Mechanical anomaly

Bearing temperature, power	Every 10 ticks	Thermal/mechanical condition, efficiency
Production load	Every ~50 ticks	Operating context
Degradation estimate	Hundreds of ticks	Long-term health
Maintenance schedule	Thousands of ticks	Planning context

3. Stateless, swappable specialists

Each processing step is a small, stateless processor wired to a neuron through an interface. Any detector can be upgraded or replaced for a particular site without touching the rest — a lab runs a simple model; a refinery plugs in a richer one behind the same interface.

5 Architecture overview: how the pieces fit together

Telemetry flows one way through increasingly intelligent stages; an auditable advisory flows out the other end:

```
FMI skid / OPC UA / MQTT / Kafka telemetry
   -> fast telemetry validation and loop state
   -> trained diagnostic finding heads
   -> economic advisory planning + fixed safety gate
   -> industrial advisory JSONL (asset, finding, confidence, economic basis)
```

The trained network is five layers of seventeen real neurons:

Layer	Size	Job, in plain language
0 — Input	—	Plant + supervisory-context boundary (OPC UA, MQTT, FMI replay, Kafka)
1 — Fast telemetry	7	Validate measurements, track interlocks, override, fast loop state
2 — Diagnostic heads	4	Trained detectors: pump wear, oscillation, sensor drift, energy
3 — Advisory planning	4	Maintenance scheduling, bounded trim, economic basis, safety gate
4 — Result	2	Emit maintenance and energy advisories as JSONL

6 The skid it supervises

The demo runs a deterministic **heated circulation and heat-exchanger skid**, simulated with an FMI 2.0 Co-Simulation FMU and driven through a real protocol gateway. It is rich enough to be a commercially testable prototype — it models the physics where the money actually leaks:

- Process and measured temperature, circulation flow, suction pressure.
- Pump speed and power, cooling-valve position, heater power.
- Vibration RMS and bearing temperature (the wear signals).

- Injectable faults: valve sticking, pump wear, temperature-sensor drift, thermal runaway.
- Deterministic local interlocks: high temperature, low flow, low suction pressure.

The model includes tank thermal inertia, pump and valve lag, pump-flow and power curves, vibration and bearing-temperature wear effects, and sensor drift/noise — so the diagnostic heads learn from realistic, physically coupled behaviour rather than a flat table.

7 The protocol boundary: OPC UA, MQTT, and PLC/SIS

OPC UA and MQTT are complementary, and the demo keeps a strict, commercially sensible boundary:

Path	Role	Authority
PLC / PID / SIS	Hard real-time control and interlocks	Always authoritative
OPC UA	Bounded local actuator command path	Only command path; tightly limited
MQTT	Telemetry and advisory distribution	Never actuates (AUTONOMOUS rejected)

There are exactly three OPC UA command nodes (cooling valve, pump speed, heater power), and the Java `MqttBridgeConfig` constructor structurally rejects an AUTONOMOUS mode — MQTT carries vibration, energy, status, and advisories, but its tags never match an actuator command tag. When a command is issued, it passes a fixed priority chain before any write happens:

```
hard interlock -> local fail-safe -> operator override -> safety mode
  -> validation/quality -> clamp -> ramp limit -> diff suppression
  -> OPC UA write -> audit
```

Command	Fail-safe default
Cooling valve	100% open (maximum cooling)
Pump speed	30%
Heater power	0% (off)

8 Signals: the common language, on many clocks

Everything the system sees becomes a **typed industrial signal**. Each declares how often it should be processed, realising the "many clocks" idea concretely:

Signal	Carries	Cadence (loop)
MeasurementSignal	Process measurements (temp, flow, pressure…)	Every loop
AlarmSignal	Plant alarms	Every loop
InterlockSignal	Hard interlock state	Every loop
OperatorOverrideSignal	Operator manual control	Every loop
ActuatorCommandSignal	Bounded actuator command	Every loop

DegradationSignal	Equipment degradation evidence	Every 10 loops
EfficiencySignal	Energy / efficiency context	Every 10 loops
SetpointSignal	Bounded setpoint recommendation	Every 10 loops
MaintenanceWindowSignal	Planned-maintenance context	Every 60 loops

Fast signals give immediate condition and safety state; slow signals carry the context that *changes the meaning* of the fast ones — without ever slowing the fast path down.

9 The four things it diagnoses

The trained layer holds four **diagnostic finding heads** — narrow, high-value detectors. Each leans on a coherent set of evidence (shown here in plain language, drawn from the model's learned weights):

1 · Pump wear and cavitation risk

Rising vibration and bearing temperature, increasing pump power, and low suction pressure / low flow point to wear or cavitation — distinguished from a mere temporary high load by the *combination* and its trend.

2 · Control-loop oscillation and tuning degradation

A valve-stiction proxy (command moves, position lags), actuator reversal density, and load-transition patterns separate genuine tuning degradation from normal transients.

3 · Temperature-sensor drift

A physical temperature model produces a residual; a large, persistent residual with no matching power or process response means the **sensor** is drifting, not the process — avoiding an unnecessary shutdown.

4 · Energy-per-unit-production deterioration

Energy consumption relative to production, combined with health risk and actuator activity, flags a loop that holds its variable correctly while quietly wasting energy.

10 The trained model and the deployable network

Bundled with the product is a checked-in reference model, `industrial-loop-guardian`. Training does not stop at a set of weights — it emits a **complete, deployable JNeopallium network**: five layers, seventeen real neurons, 39 features, 156 trainable weights and 4 biases, written into ready-to-load layer-configuration files that reference concrete runtime classes.

Generated layer	Real neurons / role
Fast telemetry	SensorNeuron, MeasurementValidatorNeuron, OscillationMonitorNeuron, …
Diagnostic heads	DegradationModelNeuron, EnergyAccountingNeuron (the four finding heads)
Advisory planning	MaintenanceSchedulingNeuron, SetpointOptimiserNeuron, EconomicBasisNeuron, SafetyGateNeuron
Result	Advisory JSONL output neurons

Each diagnostic head also records a **logical role** (e.g. `PumpHealthAndEfficiencyNeuron`, `OscillationDiagnosisNeuron`, `SensorFaultDiscriminationNeuron`, `EconomicBasisNeuron`, `SafetyEnvelopeNeuron`), the features it is allowed to use (`featureGate`), and its `ownedReasoning` — so a production reviewer can see that the logic is encapsulated inside the Jneopallium model, not hidden in glue code.

> **Read this honestly**
>
> On the bundled, deterministic reference skid the model scores near-perfectly (macro-F1 ≈ 0.9995, and only a handful of energy-head false positives out of thousands). That demonstrates the pipeline and the diagnostic separation — it is NOT a claim of real-world accuracy, which must be earned on external historian, CMMS, and energy-meter data. The Test Report states this plainly.

11 Safety by design: why it cannot trip your plant

1. **Supervisory and advisory by default.** The safety ceiling is ADVISORY. The Loop Guardian recommends; it does not control. Bounded autonomous setpoint changes require a separate, deliberately added safety case, approval workflow, and rollback path.
2. **Deterministic controls stay authoritative.** PLC/PID/SIS own fast control and hard safety. Hard interlocks, local fail-safe logic, and operator overrides take priority over anything the model suggests — structurally, in the command priority chain.
3. **MQTT cannot actuate.** The telemetry/advisory path is structurally barred from autonomous action; only the bounded OPC UA path can command, and only within clamps and ramp limits.
4. **Fail-safe on loss.** On OPC UA loss the skid applies local fail-safe values; on MQTT loss fast-loop control availability stays at 1.0. Losing the supervisor never endangers the plant.

And every advisory carries its **economic basis** and a **safety-envelope** result, so an engineer can see not just *what* is recommended but *why*, *what it is worth*, and *that it stays inside the configured envelope*.

12 Deployment topology: where it runs

Mode	Description	Typical use
Local	Single Java process	Offline replay, pilots, edge box at the plant
Cluster (HTTP)	Distributed over HTTP	Multi-asset, multi-line sites
Cluster (gRPC)	Distributed over gRPC; FPGA-capable	High-throughput fleets

Telemetry arrives through **bridges** that translate a protocol into typed signals: OPC UA measurement/alarm inputs, MQTT telemetry inputs, and a Kafka shadow stream. A site-specific input adapter converts your historian or live feeds into the same typed signals while preserving event time, so the same model runs on a laptop replay and a clustered plant alike.

13 End-to-end walkthrough: nine scenarios

The deterministic runner exercises nine scenarios and records evidence for each. A few show the architecture's character:

- **pump-wear** — degradation evidence accumulates; the pump-health head raises a maintenance advisory with an economic basis (fault evidence appears ~58 s into the run).
- **oscillation** — the oscillation head separates tuning degradation from normal load transitions.
- **temperature-sensor-drift** — the sensor-drift head attributes the anomaly to the sensor, not the process, avoiding a needless trip.
- **high-temperature-interlock** — the hard interlock writes cooling and heater fail-safe; interlock latency is ~0.1 s, owned by the deterministic layer, not the model.
- **mqtt-outage** — fast-loop control availability stays at **1.0**; advisories pause, control does not.
- **opcua-outage** — local fail-safe values are applied; the plant stays safe without the supervisor.

> **The point of the scenarios**
>
> In every case the model adds diagnosis and economic context, while the deterministic controls keep the plant safe and running. The supervisor improves the decision; it never owns the safety.

14 Glossary for non-specialists

Term	Plain-language meaning
Advisory mode	The system recommends; humans and PLCs decide. It never trips the plant.
Cavitation	Vapour bubbles forming at a pump's suction, damaging it — caused by low pressure.
Interlock	A hard, deterministic safety rule (e.g. high temp → cooling fail-safe).
FMI / FMU	A standard way to package a plant simulation; used here to drive the demo skid.
OPC UA	An industrial information/communication standard; here, the bounded command path.
MQTT	A lightweight publish/subscribe protocol; here, telemetry and advisories only.
PLC / PID / SIS	The deterministic controllers and safety system that own fast control.
Setpoint	The target value a controller aims for (e.g. desired temperature).
Stiction	Static friction in a valve that makes its position lag its command.
Supervisory layer	A layer that reasons across loops and timescales, above the controllers.
Economic basis	The money value the system attaches to a finding (e.g. avoided shutdown).

Jneopallium Industrial Loop Guardian · FMI Skid Demo. Architecture article for technical and non-technical readers. Supervisory above PLC/PID/SIS. Safety mode: ADVISORY. License: BSD 3-Clause.